EZ-USB™ FX3 SDK Release Notes

About this document

Scope and purpose

Thank you for your interest in the EZ-USB™ FX3 software development kit (SDK). This SDK supports all products in the FX3 family (FX3, CX3, SD3, FX3S and FX2G2), and is an update to the previously released SDK 1.3.4 version.

Intended audience

This document is primarily intended for those using the EZ-USB™ FX3 software development kit.

Table of contents

About this document **1**
Table of contents **1**
1 Introduction **4**
2 Release components **5**
3 Tool information **6**
3.1 Features 6
3.1.1 Firmware features 6
3.1.2 Development tool features 6
3.1.3 USB suite features 6
3.2 GPIF II designer features 7
3.3 Changes from FX3 SDK 1.3.4 to FX3 SDK 1.3.5 7
3.3.1 Device APIs 7
3.3.2 DMA APIs 7
3.3.3 USB driver and APIs 7
3.3.4 Serial peripheral driver and APIs 7
3.3.5 GPIF/PIB driver and APIs 7
3.3.6 Storage driver and APIs 7
3.3.7 Boot API and examples 8
3.3.8 Firmware examples 8
3.3.9 Eclipse IDE 8
3.4 Changes from FX3 SDK 1.3.3 to FX3 SDK 1.3.4 8
3.4.1 Device APIs 8
3.4.2 DMA APIs 8
3.4.3 USB driver and APIs 8
3.4.4 Serial peripheral driver and APIs 9
3.4.5 GPIF/PIB driver and APIs 9
3.4.6 Storage driver and APIs 9
3.4.7 CX3 API support 9
3.4.8 Firmware examples 9
3.4.9 Eclipse IDE 9
3.5 Changes from FX3 SDK 1.3.1 to FX3 SDK 1.3.3 10
3.5.1 Device support 10
3.5.2 Utility functions 10

Table of contents

3.5.3 DMA APIs 10
3.5.4 USB driver and APIs 10
3.5.5 Storage driver and APIs 10
3.5.6 Debug APIs 11
3.5.7 UART APIs 11
3.5.8 I2C / SPI APIs 11
3.5.9 Boot API and examples 11
3.5.10 Firmware examples 11
3.5.11 Toolchain changes 12
3.5.12 Documentation changes 12
3.5.13 Packaging changes 12
3.6 Changes from FX3 SDK 1.3 to FX3 SDK 1.3.1 12
3.6.1 Device support 12
3.6.2 Build changes 12
3.6.3 DMA driver and APIs 12
3.6.4 USB driver and APIs 13
3.6.5 Storage driver and APIs 13
3.6.6 Debug APIs 13
3.6.7 Firmware examples 13
3.6.8 CX3 APIs and examples 13
3.7 Changes from SDK 1.2.3 to SDK 1.3 14
3.7.1 Device support 14
3.7.2 Device APIs 14
3.7.3 Storage API and examples 14
3.7.4 USB driver and APIs 14
3.7.5 Beta CX3 APIs and examples 14
3.7.6 PIB driver and APIs 15
3.7.7 2-Stage boot APIs 15
3.7.8 Build scripts and utilities 15
3.7.9 Example applications 15
3.7.10 Documentation 15
3.8 Changes from SDK 1.2.2 to SDK 1.2.3 16
3.8.1 Eclipse integration 16
3.8.2 General firmware framework 16
3.8.3 USB driver and API 16
3.8.4 Programmer's manual 16
3.9 Changes from SDK 1.2.1 to SDK 1.2.2 16
3.9.1 SDK installation path 16
3.9.2 Boot firmware library and API 16
3.9.3 General firmware framework 16
3.9.4 USB driver and API 17
3.9.5 GPIF driver and API 17
3.9.6 Example build settings 17
3.10 Changes from SDK 1.2 to SDK 1.2.1 17
3.10.1 General firmware framework 17
3.10.2 USB drivers and APIs 17
3.10.3 DMA driver and APIs 18
3.10.4 GPIO driver and APIs 18
3.10.5 Two-Stage boot firmware library 18
3.10.6 Firmware examples 18

Table of contents

3.10.7 Firmware binary converter 18
3.10.8 SDK documentation 19
3.11 Changes from SDK 1.1.1 to SDK 1.2 19
3.11.1 Serial peripheral drivers 19
3.11.2 Device APIs 19
3.11.3 GPIF driver and APIs 19
3.11.4 USB driver and APIs 19
3.11.5 Build settings 19
3.11.6 Two-Stage boot firmware library 20
3.11.7 Firmware examples 20
3.12 Changes from SDK 1.1 to SDK 1.1.1 20
3.12.1 General changes 20
3.12.2 USB driver and API changes 20
3.12.3 I2C driver changes 21
3.12.4 GPIF driver and API changes 21
3.12.5 Debug API changes 21
3.12.6 Example application changes 21
3.12.7 Boot firmware changes 21
3.13 Changes from SDK 1.0.1 to SDK 1.1 21
3.13.1 General changes 21
3.13.2 USB driver and APIs 22
3.13.3 I2C driver and APIs 22
3.13.4 GPIO 22
3.13.5 Two-Stage boot firmware library 22
3.13.6 Firmware examples 22
3.14 Changes from SDK 1.0 to SDK 1.0.1 22
3.14.1 Linux support 22
3.14.2 USB driver 23
3.14.3 GPIF driver and APIs 23
3.14.4 I2C driver and APIs 23
3.14.5 UART driver and APIs 23
3.14.6 ELF to boot image converter 23
3.14.7 Two-Stage boot firmware library 23
3.14.8 Firmware examples 23
4 Known problems and limitations of FX3 SDK 1.3.5 24
5 Technical Support 27
6 Additional Information 28

1 Introduction

The EZ-USB™ FX3 controller is a general-purpose integrated USB 3.0 SuperSpeed controller with a built-in programmable interface (GPIF II) and support for accessing a set of serial peripherals.

The EZ-USB™ FX3S (CYUSB3035) controller is an extension to the EZ-USB™ FX3 device, which adds the ability to control up to two SD/MMC/SDIO peripherals to the FX3 feature set.

The EZ-USB™ CX3 (CYUSB3065) controller is an extension to the EZ-USB™ FX3 device, which adds the ability to interface with and perform uncompressed video transfers from image sensors implementing the MIPI CSI-2 interface.

The EZ-USB™ FX2G2 (CYUSB2014) controller is a version of FX3 that does not support the USB 3.0 SuperSpeed PHY. All other features of the FX3 device are supported by the FX2G2 controller.

The EZ-USB™ SD3 (CYUSB3025) controller is a USB 3.0 to SD/MMC/SDIO bridge device based on the FX3 architecture. This part does not support the GPIF II or MMC Slave interface.

See the Getting Started with FX3 SDK document for block diagrams and more information about these devices.

The FX3 SDK enables users to make use of the device features for each of these devices effectively.

FX3 and associated devices have an embedded ARM9 microcontroller, and the SDK contains a set of flexible software modules that enable the effective use of all device features. The primary component of the SDK is a set of APIs that can be used by programmers to configure and control the data paths through the device as well as peripherals connected to the device.

The FX3 SDK consists of the following major components:

1. FX3 Firmware stack with the API library and sample firmware examples.
2. Eclipse IDE and GNU Toolchain.
3. Documentation – API documentation, Programmer's manual, and Build instructions.

2 Release components

This 1.3.5 release of the FX3 SDK includes all the standard FX3 SDK components:

- FX3 Firmware drivers and APIs for device initialization, configuring USB, configuring DMA, configuring the GPIF II interface, debug logging, and configuring the serial interfaces.
- FX3S firmware driver and API for managing SD/eMMC/SDIO peripherals connected to the device.
- CX3 APIs for interfacing with image sensors using the MIPI-CSI2 interface.
- FX3 Boot Firmware drivers and APIs for device initialization, configuring USB and configuring the serial interfaces.
- FX3 firmware sources and build scripts.
- Sample FX3 firmware examples in the form of Eclipse projects (e.g., USB data loopback example, USB data source sink examples, serial interface examples (I2C, UART, SPI, and GPIO), USB Video Class example, Slave FIFO example, and FX3 boot source (I2C EEPROM and SPI FLASH) programming example).
- FX3S firmware examples that demonstrate USB mass storage class implementation, GPIF to storage access, File System integration, and SDIO peripheral access.
- CX3 firmware examples that demonstrate USB Video class implementation using uncompressed 1080p and 720p streams at 30 and 60 frames per second respectively.
- A MIPI CSI-2 interface configuration tool that simplifies UVC implementations using CX3.
- FX3 SDK Getting Started guide, API documentation, and programmer's manual
- GPIF II Designer (Supported only under Windows)
- Eclipse IDE with GNU ARM C/C++ Development support plug-in
- GNU Arm® Toolchain
- Infineon Software License Agreement

3 Tool information

3.1 Features

3.1.1 Firmware features

- Supports USB - SuperSpeed, Hi-Speed, and Full Speed device operation.
- Supports USB Hi-Speed and Full Speed Host/OTG operation.
- Tested with the Intel, Renesas, Fresco, Via Labs, ASMedia, and TI XHCI host controllers.
- Supports configuring GPIF II interface
- Supports pass-through data traffic between USB host and GPIF II interface at USB 2.0 and 3.0 speeds.
- Supports controlling SD/eMMC/SDIO peripherals and performing data transfers.
- Supports UART configuration and data communication.
- Supports I^2C (master mode only) configuration and data communication.
- Supports SPI (master mode only) configuration and data communication.
- Supports Debug logging
- Supports simple and complex mode GPIO access.
- Sample firmware examples
- Detailed firmware documentation can be found under the doc/firmware folder.

3.1.2 Development tool features

- Arm® GNU Toolchain comprising GNU compiler (gcc), GNU binary utilities (assembler, linker), and GNU debugger (gdb)
- Full-featured Eclipse IDE
- GNU Arm® C/C++ Development support plug-in for Integrated firmware development and debugging.
- Supports Windows, Linux, and Mac OS platforms.
- User documentation on the Eclipse IDE can be found under the *doc/firmware* folder.

3.1.3 USB suite features

- Generic USB device driver (CyUsb3.sys) for Windows platforms.
- C++ and C# based convenience API libraries for performing USB data transfers through the CyUsb3.sys driver.
- LibUsb based convenience API library for performing USB data transfers on Linux and Mac platforms.
- USB Control Center Windows application for device programming and data transfers.
- cyusb_linux GUI application for device programming and data transfers.
- Bulkloop and Streamer Windows applications to exercise loop back data transfers and to perform transfer through-put testing.
- Console-based applications for device programming and data transfers on Linux and Mac platforms.
- Windows USB Suite documentation can be found under the *doc/SuiteUSB* folder. For Linux and Mac platforms, documentation can be found under the *doc* folder in the cyusbsuite package.

3.2 GPIF II designer features

- Interface customization feature supporting a list of five standard GPIF II interfaces.
- Custom interface definition and state machine design capability
- Simulation feature that shows the behavior of input and output signals when traversing a path in the state machine.
- User documentation on the GPIF II Designer can be found under the *doc/GPIFIIDesigner* folder.

3.3 Changes from FX3 SDK 1.3.4 to FX3 SDK 1.3.5

3.3.1 Device APIs

1. Updated `CyU3PDebugPrint()` to release mutex lock in case of DMA error.
2. Updated `CyU3PDeviceInit()` to include SYSMEM redundancy initialization. This prevents SRAM corruption issue noticed after a device warm reset on devices that have stuck-at faults in the SRAM array.

3.3.2 DMA APIs

1. Updated `CyU3PDmaChannelGetBuffer()` and `CyU3PDmaMultiChannelGetBuffer()` APIs to return error if the buffer has already been committed using `CyU3PDmaChannelCommitBuffer` or `CyU3PDmaMultiChannelCommitBuffer` APIs.

3.3.3 USB driver and APIs

1. Updated `CyFx3Usb3LnkSetup` API to set PendingHPTimer duration to 10 µs and U1 exit LFPS duration to 1 µs for USB 3.1 compliance tests.
2. Updated `CyU3PConnectState` API to check for CY_U3P_UIB_FORCE_FS bit while trying for non-SuperSpeed connection.
3. Added a condition in `CyU3PUibIntHandler()` to truncate LFPS transmission by FX3 on detecting Warm Reset LFPS from Host.

3.3.4 Serial peripheral driver and APIs

1. Updated the `CyU3PI2cSetConfig()` function to configure the device for maximum drive strength when I2C fast mode plus bit rate is selected.
2. `CyU3PSpiTransmitWords()` updated to fix data corruption issue during SPI transmission.
3. `CyU3PSpiSetTxFlushWaitPeriod()` added to insert an additional delay before the TX path gets disabled in the `CyU3PSpiTransmitWords()` function.

3.3.5 GPIF/PIB driver and APIs

1. Updated `CyU3PGpifDisable()` to disable interrupts while disabling the GPIF interface

3.3.6 Storage driver and APIs

1. Added CY_U3P_SIB_IDLE_TIMER events to commit pending writes to SIB. This prevents data loss if the host abruptly stops data transmission.

3.3.7 Boot API and examples

1. Updated `CyFx3BootDeviceInit` to include SYSMEM redundancy initialization. This prevents SRAM corruption issue noticed after a device warm reset on devices that have stuck-at faults in the SRAM array.
2. Updated `CyFx3BootI2cSetConfig()` function to configure the device for maximum drive strength when I^2C fast mode plus bit rate is selected.
3. `CyFx3BootI2cSendStopCondition` and `CyFx3BootI2cErrorRecovery` APIs were added to handle I^2C errors.
4. Added a condition in `HandleUsbLnkInterrupt()` to truncate LFPS transmission by FX3 on detecting Warm Reset LFPS from Host.

3.3.8 Firmware examples

1. Mass storage example – corrected the sequence in which DMA channel reset is being performed in case of SIB data error event.
2. Disabled UART flowCtrl setting to enable testing of firmware examples in FX3 Superspeed Explorer kit.
3. Synchronous Slave FIFO example – increased DMA buffer size and burst to increase observed throughput.

3.3.9 Eclipse IDE

1. CX3 Configuration tool updated to support parameter generation for multi-format UVC configuration.

3.4 Changes from FX3 SDK 1.3.3 to FX3 SDK 1.3.4

The FX3 SDK 1.3.4 release includes the following changes on top of the 1.3.3 version. Please refer to the FX3 API guide for more details of the specific API level changes.

3.4.1 Device APIs

1. Updated the `CyU3PSysCheckSuspendParams()` API to allow device suspend with USB bus activity wake-up even in cases where the USB block is powered from Vbatt instead of Vbus.
2. Updated the `CyU3PSysCheckSuspendParams()` API to use the correct settings for UART CTS based wakeup in all IO configurations.

3.4.2 DMA APIs

1. Updated the CY_U3P_DMA_TYPE_AUTO_SIGNAL channel implementation to support consume event notifications.
2. Added a new API called `CyU3PDmaSocketClearInterrupts()` to simplify low-level DMA implementation. Refer to the USBBulkLoopLowLevel firmware example for usage details.
3. Updated the `CyU3PDmaChannelSuspendUsbConsumer()` implementation to minimize the time for which other endpoints are halted while handling USB-HS control IN transfers.

3.4.3 USB driver and APIs

1. Implemented firmware work-around to prevent USB 3.0 recovery failures that are seen on some devices.
2. Corrected implementation of `CyU3PUsbEPSetBurstMode()` for optimal transfer performance.
3. Added a new USB event (CY_U3P_USB_EVENT_LMP_EXCH_FAIL) to provide notification in cases where the Port.Capability or Port.Configuration LMP handshake fails during USB connection.
4. Added the `CyU3PUsbEpEvtControl()` API so that different interrupt notifications can be registered for each USB endpoint.
5. Added the `CyU3PUsbEpSetPacketsPerBuffer()` API to speed up data transfer on USB-HS isochronous endpoints with arbitrary maximum packet size settings.

6. Added the `CyU3PUsbSetEpSuspDisableMask()` API to allow users to select endpoints, which should not be halted temporarily while USB-HS control IN transfers are handled.
7. Added the `CyU3PUsbLinkComplianceControl()` API to allow support for USB 3.0 link compliance mode to be disabled. This is a useful hook for debugging enumeration failures.
8. Updated USB host API implementation to avoid possibilities of timeouts during fast USB connect/disconnect sequences.

3.4.4 Serial peripheral driver and APIs

1. Added the `CyU3PI2cSendStopCondition()` API, which can be used to drive out an I^2C stop condition to recover from I^2C errors in multi-master use cases.
2. Added the `CyU3PI2sEnableExternalMclk()` API, which can be used to configure the I2S block to run with an externally provided master clock.
3. Added the `CyU3PSpiTransferWords()` API, which supports concurrent SPI transmit and receive operation in register mode. Previously, concurrent transmit and receive was only supported in DMA mode.
4. Updated `CyU3PSpiTransmitWords()` and `CyU3PSpiReceiveWords()` implementation to speed up data transfers.
5. Updated `CyU3PUartTransmitBytes()` and `CyU3PUartReceiveBytes()` implementation to speed up data transfers.

3.4.5 GPIF/PIB driver and APIs

1. Added the `CyU3PPibDllConfigure()` API, which allows the DLL associated with the GPIF block to be configured where required.

3.4.6 Storage driver and APIs

1. Updated `CyU3PSibEraseBlocks()` API to support erase operation on boot partitions of eMMC devices.
2. Updated `CyU3PSibVendorAccess()` API to remove the pad bytes which were present in the response buffer.

3.4.7 CX3 API support

1. Updated the GPIF II state machine used to collect the data from the MIPI CSI-2 bridge interface to reduce the blanking interval required between frames.

3.4.8 Firmware examples

1. Updated all USB firmware examples to report bcdUSB as USB 3.1.
2. Removed the USBBulkStreams firmware example as driver support is not available on Windows.
3. Updated the CX3 firmware examples to adapt to the new GPIF II state machine used.
4. Updated the Mass Storage firmware examples to periodically commit pending write operations on the SD/eMMC media.
5. Disabled USB 3.0 LPM support in most of the generic USB examples which do not have a well-defined data transfer protocol. LPM support is still enabled and controller properly in real use cases like UVC, MSC and HID applications.

3.4.9 Eclipse IDE

1. Updated GNU-Arm® Eclipse plug-ins to a newer version.

3.5 Changes from FX3 SDK 1.3.1 to FX3 SDK 1.3.3

The FX3 SDK 1.3.3 release included the following changes on top of the 1.3.1 version.

3.5.1 Device support

1. Added support for the FX2G2 devices (CYUSB2014, CYUSB2013 and CYUSB2011) in the firmware library. These are USB 2.0 only variants of the FX3 device. All FX3 examples will work with the FX2G2 devices in USB 2.0 mode.
2. Added support for the Benicia (CYWB0263) and Bay (CYWB0163) devices. These are equivalents of the FX3S and FX2S devices in a different package.

3.5.2 Utility functions

1. Provided a common implementation for the `CyU3PBusyWait` and `CyFx3BootBusyWait()` APIs, which is calibrated to have less than 2% error across all firmware build configurations and which adjusts for CPU frequency changes.
2. Updated the `CyU3PMemCopy` and `CyU3PMemCopy32` functions to be capable of copying memory between overlapping regions (similar to the memmove library function).
3. Update the `CyU3PMemAlloc` and `CyU3PDmaBufferAlloc` implementations to support memory leak detection and memory corruption detection (at the time of a free call). These features are left disabled by default and will increase the memory overheads of each allocated memory block when enabled.

3.5.3 DMA APIs

1. Added a pair of new signature fields into each DMA channel structure. These signatures are used to check for validity of the DMA channel handle when any channel API is called. This change prevents potential issues due to using a DMA channel before it is created or after it is destroyed.

3.5.4 USB driver and APIs

1. Implemented firmware level work-arounds to resolve FX3 device errors leading to data corruption when operating under poor USB 3.0 link conditions. Please refer to the GpifToUsb firmware example and the FX3 SDK Troubleshooting Guide for more details.
2. Implemented a firmware solution to prevent data corruption due to concurrent BULK-IN and Control-IN transfers on a Hi-Speed link.
3. Added proper handling for the USB 2.0 LPM-L1 power mode transition. The `CyU3PUsbLPMDisable()` API can be used to control LPM-L1 entry as well as the USB 3.0 U1/U2 entry.
4. Fixed a firmware defect leading to U1 exit LFPS duration which is out of spec.
5. Fixed a firmware defect resulting in the USB 3.0 resume (U3 > U0 transition) event notification not being provided correctly.
6. Fixed a firmware defect resulting in the suppression of ITP events while USB control requests (EP0) are being handled.
7. Added a new API to turn off USB 3.0 Spread Spectrum Clocking on the FX3 device. See `CyU3PUsbSSCDisable` in the API guide. Disabling SSC support can improve USB link stability in some cases.

3.5.5 Storage driver and APIs

1. Added the capability to configure the data bus width for SD/MMC interface to 1 bit.

3.5.6 Debug APIs

1. Added a new API called `CyU3PDebugSetTimeout()` to set the timeout period to be used when using the Debug Log functionality.

3.5.7 UART APIs

1. Added a new API called `CyU3PUartSetTimeout()` to set the timeout period to be used when using the UART transmit and receive APIs.
2. Added a new UART event CY_U3P_UART_EVENT_RX_DATA, which is used to provide notification of presence of data in the receive FIFO. In addition, updated the `CyU3PUartSetConfig()` API to allow setting a callback function in register mode, so that this event can be used.

3.5.8 I2C / SPI APIs

1. Added APIs called `CyU3PI2cSetTimeout()` and `CyU3PSpiSetTimeout()` to control the timeout applied to the register mode data transfer APIs. These APIs can be used to override the default timeout setting.

3.5.9 Boot API and examples

1. Added DMA and PIB (GPIF and PMMC) support to the FX3 boot library. See the API reference guide for more details.
2. Added new APIs (`CyFx3BootUsbGetLinkPowerState`, `CyFx3BootUsbSetLinkPowerState`, `CyFx3BootUsbSigResume`, and `CyFx3BootUsbHandleEvents`) to manage USB low power mode transitions.
3. Added debug logging support through the UART using the `CyFx3BootUartPrintMessage()` API. This can be used in both DMA and register mode of UART operation.
4. Added a new example `BootGpifDemo` which uses the DMA and PIB API to implement a copy of the GpifToUsb firmware example.
5. Added a new example (BootLedBlink) that shows how the boot library can be used to control FX3 device GPIOs.

3.5.10 Firmware examples

1. Updated the build settings for all firmware examples to use a new project-specific build variable called FX3SDKVERSION, which selects the FX3 SDK library version to be linked with. The old project settings continue to work as the latest firmware libraries are available under their old paths. However, use of the new project template is recommended as it allows easy migration from one library version to another.
2. Updated the GpifToUsb firmware example to demonstrate the use of the new firmware work-around to prevent USB data corruption.
3. Updated the USBVideoClass example to work properly at Hi-Speed on all standard PC operating systems (Windows, Linux, and Mac OS).
4. Updated all CX3 firmware examples to support additional UVC class requests that are required by the UVC driver on Linux.
5. Added a new PibSlaveDemo example that shows how FX3S can be configured to work as a co-processor (slave) device that provides peripheral expansion capability to an external processor.
6. Added a new Fx2g2UvcDemo example that implements an UVC application with only USB 2.0 support. This can be used as a reference implementation for applications targeted at USB 2.0 only devices like FX2G2.
7. Added a new SRAMMaster example which shows how the GPIF II can be used in master mode. This example works on the CYUSB3KIT-003 kit and demonstrates how the FX3 can access an asynchronous SRAM device.

3.5.11 Toolchain changes

1. Moved to a more up-to-date version of the GNU C Compiler (4.8.1) from Mentor Graphics.
2. Added support for firmware development and debug on Mac OS platforms (tested on OS X Yosemite) using an Eclipse IDE and the Yagarto GNU ARM tool-chain.
3. Provided a new CX3 configuration utility which can generate USB descriptors and UVC streaming source code in addition to the MIPI CSI-2 configuration.

3.5.12 Documentation changes

1. Moved all Eclipse IDE usage information into a new EZUsbSuite User Guide document.
2. Provided supported device list as part of Getting Started with FX3 SDK document.
3. Updated the device and firmware description sections in the FX3 programmers' manual.

3.5.13 Packaging changes

1. Provided FX3 firmware libraries and headers from all previous releases (starting with 1.2.1) under the fw_lib and boot_lib folders inside the SDK. The libraries in these paths have been renamed with a lib prefix, so that the "–l" linker option can be used to specify the input libraries. All libraries are still available in their original paths (u3p_firmware) so as to maintain compatibility with existing Eclipse workspaces and projects.
2. Moved the build scripts out of the firmware folder into the fw_build/fx3_fw and fw_build/boot_fw folders. All scripts are still available in their original paths as well, so as to maintain compatibility with existing Eclipse workspaces and projects.
3. Provided all FX3 firmware sources in the form of a ZIP archive within the SDK installation.
4. Provided pres-built binaries of OpenOCD 0.8.0 that support debugging using the Cypress CY7C65215 part which is built-in on the CYUSB3KIT-003 boards.

3.6 Changes from FX3 SDK 1.3 to FX3 SDK 1.3.1

The FX3 SDK 1.3.1 version includes the following changes on top of the 1.3 version.

3.6.1 Device support

1. Full support for the CX3 device in the firmware library. The CX3-specific features are only available in the full firmware library, and not part of the boot library.

3.6.2 Build changes

1. Changed the build options for the firmware library to obtain reduced memory footprint. These changes compensate for the 10 KB of additional code (defect fixes and new device support) that was added in SDK 1.3.

3.6.3 DMA driver and APIs

1. Fixed a defect in the implementation of the `CyU3PDmaMultiChannelDiscardBuffer()` API for Many-to-One manual DMA channels.
2. Reverted a change made in SDK 1.3 to release the lock on a DMA channel before calling the callback function. The lock on the channel is now held while the callback function is invoked. This will allow APIs that use the same channel to work faster when called from the callback routine.
3. Added a new API called `CyU3PDmaMulticastSocketSelect()` to select the active consumer sockets dynamically on a multicast DMA channel.

3.6.4 USB driver and APIs

1. Fixed a USB start-up defect which caused the device to re-enumerate when loading an application on top of a SDK 1.2.3 or older version of the bootloader.
2. Fixed the `CyU3PUsbStall` API to compulsorily send an ERDY TP so that the host can identify that there is a STALL condition. Previously, the ERDY was only sent if the endpoint was in a flow-controlled state.

3.6.5 Storage driver and APIs

1. Freed up two additional DMA sockets (CY_U3P_SIB_SOCKET_4 and CY_U3P_SIB_SOCKET_5) for applications using the FX3S part.
2. Added a new field called `lvGpioState` to the CyU3PSibIntfParams_t structure. This field defines the polarity of the GPIO that controls the SD port voltage switch.
3. Fixed a defect in the calculation of erase unit size for SD cards of 64 GB and greater capacities.

3.6.6 Debug APIs

1. Fixed a defect in the `CyU3PDebugSysMemInit()` API implementation, which prevented the logs from being added to the memory buffer.

3.6.7 Firmware examples

1. Updated the UsbBulkSourceSink application to restart sending data on the source (IN) endpoint after completion of a CLEAR_FEATURE(EP_HALT) command.
2. Updated all firmware example projects to remove the "**Create Extended Listing**" option in the default build settings. This option should be added manually by the user if required.
3. Added a new example (FX3SRaid0) that demonstrates the use of FX3S APIs to implement a RAID-0 disk.

3.6.8 CX3 APIs and examples

1. Updated APIs as part of the CX3 firmware library (cyu3mipicsi.a) that provides the MIPI-CSI interface support. FX3/FX3S users do not need to link with this new library and can use only the changes listed in other categories.
2. Updated examples that demonstrate various features of the CX3 API using Aptina AS0260 sensor and Omnivision OV5640 sensor. Sensor functionality is implemented through sensor-specific libraries (cy_as0260.a and cy_ov5640.a).
3. The CyU3PMipicsiCfg_t structure used for configuring the MIPI-CSI interface has changed. The order of parameters in the structure has changed and one additional parameter has been added.
4. A preliminary release, for a CX3 MIPI CSI-2 interface configuration generation tool, has been provided within the EZ-USB™ Suite Eclipse IDE. This tool simplifies creation of the CyU3PMipicsiCfg_t structure, based on image stream and sensor settings. Instructions for use of the tool are provided in the Getting Started with FX3 guide.
5. The fixed function GPIF II interface is updated for improved stability.

3.7 Changes from SDK 1.2.3 to SDK 1.3

The 1.3 release of the FX3 SDK has the following changes from the 1.2.3 version.

3.7.1 Device support

1. Added support for the EZ-USB™ FX3S, SD3 and SD2 device families in the SDK libraries. The FX3S family adds the capability to communicate with SD/MMC/SDIO peripherals to the FX3 device. The SD3 and SD2 families are USB to peripheral bridge devices that support the SD/MMC/SDIO devices in addition to the serial peripheral (UART, SPI, I2C and I2S) interfaces.
2. Added BETA support for the EZ-USB™ CX3 device family in the SDK libraries. The CX3 family adds the capability to interface with Image Sensors using the MIPI-CSI2 interface.

3.7.2 Device APIs

1. Added new fields called s0Mode and s1Mode to the CyU3PIoMatrixConfig_t structure passed to the `CyU3PDeviceConfigureIoMatrix` API. FX3 users should set these fields to the value CY_U3P_SPORT_INACTIVE. FX3S users can select 4-bit or 8-bit wide storage port operation through these parameters.
2. Added a new lppMode definition called CY_U3P_IO_MATRIX_LPP_NONE. This mode should be chosen for FX3S designs which use the S1 storage port in 8-bit wide mode.
3. Fixed an error in the `CyU3PSysEnterSuspendMode()` API that could get the device stuck when used with USB bus activity as the wake-up source.
4. Changes to allow the watchdog to be left running when transferring control from the 2-stage boot firmware to the full firmware application. If the watchdog is not explicitly disabled prior to initiating the control switch, it will be left running and needs to be cleared when the new firmware application starts execution.

3.7.3 Storage API and examples

1. Added a new firmware library (cyu3sport.a) that provides the storage drivers and APIs for the FX3 device. FX3 users do not need to link with this new library and can use only the changes listed in other categories.
2. Added examples (FX3SMassStorage, GpifToStorage, FX3SFileSystem, FX3SSdioUart) that demonstrate various features of the FX3S API.

3.7.4 USB driver and APIs

1. Fixed a driver defect that could prevent the device from staying in the USB 3.0 compliance state for doing electrical compliance testing.
2. Fixed a defect in the `CyU3PUsbDoRemoteWakeup()` API that would cause the device to drop off the bus after signaling remote wakeup.
3. Added support for detecting USB connections based on the VBatt supply (as a replacement to the VBus supply). Please refer to the `CyU3PUsbControlVbusDetect()` API for details.

3.7.5 Beta CX3 APIs and examples

1. Added a new firmware library (cyu3mipicsi.a) that provides the MIPI-CSI interface support. FX3/FX3S users do not need to link with this new library and can use only the changes listed in other categories.
2. Added examples that demonstrate various features of the CX3 API.

3.7.6 PIB driver and APIs

1. Added support for configuring the Processor Interface Block (PIB) port on FX3/FX3S as an MMC slave in addition to the default GPIF mode. Only one of the MMC slaves or GPIF modes can be used at a time. Please refer to the `CyU3PPibSelectMmcSlaveMode` API for configuring the PIB as an MMC slave.
2. Added fixed function GPIF interfaces (8-bit, 16-bit and 24-bit) for the CX3 parts. The CX3 parts do not contain a user configurable GPIF interface.

3.7.7 2-Stage boot APIs

1. Added the `CyFx3BootGpioOverride` and `CyFx3BootGpioRestore` APIs to allow run-time functionality overrides on the FX3/FX3S device IOs.
2. Changes to allow the Watchdog to run when switching control from the boot firmware to the full firmware or vice versa. This allows the device to recover in cases where the new firmware fails to start-up correctly (due to configuration/programming errors).
3. Fixed SPI configuration errors that restricted the interface clock to half of the user specified frequency.
4. Reduced the polling delays in the I2C/SPI WaitForXfer APIs to allow better transfer speeds.
5. Moved the run-time stacks used by the firmware to the DTCM region and allocated a larger stack (4 KB) for the default execution mode (System mode).
6. Changed the USB VID/PID used by the USB boot example to the same as that used by the main FX3 examples.

3.7.8 Build scripts and utilities

1. Provided new linker scripts (cyfx3_256k.ld and fx3_256k.ld), which can be used when building applications for FX3 parts having only 256 KB of RAM.
2. Changed the default i2cConf value used by the elf2img converter program to 0x1C. This value corresponds to the use of I2C EEPROMs with linear addressing (slave address incremented after each 64 KB) with a 400 KHz. This value works with the I2C EEPROM device that is in place on the latest FX3/FX3S DVK boards.

3.7.9 Example applications

1. Added a new example application (cyfxusbuart) that demonstrates the implementation of a CDC-ACM compliant USB to UART bridge device.
2. Updated the USBBulkLoopAuto example to demonstrate the use of the CyU3PSysEnterSuspendMode() API.

3.7.10 Documentation

1. Updated the format of the API reference guide (*FX3APIGuide.pdf*). This document is now generated using a different tool, and will be organized differently.

3.8 Changes from SDK 1.2.2 to SDK 1.2.3

The FX3 SDK 1.2.3 release has the following additional changes on top of the 1.2.2 release version.

3.8.1 Eclipse integration

1. The FX3 API help has been integrated into the Eclipse development environment. All of the API documentation in the header files has been re-formatted to enable this change.

3.8.2 General firmware framework

1. Changed the return type of the CyU3PDmaBufferFree function from void to int, to enable the function to flag errors. This change will cause existing application to break during compilation. The CyU3PDmaBufferFree implementation from the cyfxtx.c files in the new SDK should be used to fix this.
2. Fixed a bug that could cause the FX3 device to get stuck when placed in Suspend mode through the CyU3PSysEnterSuspendMode API.

3.8.3 USB driver and API

1. Updated the USB driver to re-attempt a USB 3.0 connection on the very first USB 2.0 reset, instead of waiting for a USB 2.0 reset following a control request. This enables the device to consistently enumerate on USB 3.0 when used with a host stack that does not follow the normal USB initialization sequence.

3.8.4 Programmer's manual

1. Added description on how the Olimex Arm® USB OCD debug probe can be used to debug FX3 firmware. See section 12.2.1 of the programmer's manual for details.

3.9 Changes from SDK 1.2.1 to SDK 1.2.2

The FX3 SDK 1.2.2 release has the following changes on top of the 1.2.1 release version.

3.9.1 SDK installation path

1. The SDK installer has been updated to use the "C:\Program Files" directory as the default installation path, instead of "C:\Cypress". This change is only applicable for a fresh SDK installation and not for updates from a previous version of the SDK.

3.9.2 Boot firmware library and API

1. Updated the USB driver to handle aborted USB control requests cleanly.
2. Added a new parameter to the `CyFx3BootUsbSetClrFeature()` API to indicate whether the USB device has been configured or not.
3. Fixed a couple of control request handling errors in the Fx3BootAppGcc example application.

3.9.3 General firmware framework

1. Added a new API called `CyU3PSysEnterStandbyMode()` to place the FX3 device in low power standby mode.
2. Included more error checks around all ThreadX OS services that are used by the FX3 library and by user applications.

3.9.4 USB driver and API

1. Updated the USB block clocking scheme to fix intermittent failures in USB Hi-Speed enumeration, particularly when FX3 is clocked using a 26 MHz or 52 MHz clock input.
2. Fixed a USB driver issue, which could cause the device to re-enumerate occasionally due to USB 3.0 link errors that accumulate over a long period of time.
3. Enabled new USB events (***CY_U3P_USB_EVENT_VBUS_VALID*** and ***CY_U3P_USB_EVENT_VBUS_REMOVED***) to provide notification of the VBUS signal being turned on or off.
4. Added a new API called `CyU3PUsbEPSetBurstMode()` to allow the FX3 device to combine data from multiple DMA buffers into a single transfer burst on the USB IN endpoint. Using this API can improve the data transfer performance on burst enabled endpoints.
5. Added new APIs (`CyU3PUsbGetEpSeqNum` and `CyU3PUsbSetEpSeqNum`) to save and restore the sequence number on USB 3.0 endpoints.
6. Added a new API called `CyU3PUsbControlVBusDetect` to select whether VBUS detection should be done by the FX3's USB driver or not. If VBUS detection in the driver is turned off, it is expected that the CyU3PConnectState() API will be called at appropriate times based on an external means of VBUS detection.
7. Added a new API called `CyU3PUsbControlUsb2Support` to disable the USB 2.0 device PHY on the FX3 device. This API is only useful in designs where the FX3 device is being used only in SuperSpeed mode, and the USB 2.0 operation is handled by an external controller. In such a case, the ***CY_U3P_USB_EVENT_USB3_LNKFAIL*** event will provide notification that the 3.0 connection has failed.

3.9.5 GPIF driver and API

1. Added support for early notification of GPIF state machine interrupts in the interrupt context itself. Added a new API called `CyU3PGpifRegisterSMIntrCallback` to register the callback function to be called for this notification.

3.9.6 Example build settings

1. The build settings for the API libraries have been updated to place each function in a different code section. The linker parameters in all the Eclipse projects have been updated to add the "-Wl,--gc-sections" parameter to take advantage of this change. The use of this option can help reduce the memory footprint of FX3 applications.

3.10 Changes from SDK 1.2 to SDK 1.2.1

The FX3 SDK 1.2.1 release has the following additional changes on top of the 1.2 release version.

3.10.1 General firmware framework

1. Re-organized the firmware modules so as to reduce the memory footprint of common applications (not using USB OTG or multicast DMA channels) by another 20 KB.

3.10.2 USB drivers and APIs

1. Added a new API called `CyU3PUsbEnableEPPrefetch()` to enable the DMA prefetch settings that are required to prevent data corruption due to endpoint underrun errors. This API needs to be called after `CyU3PUsbStart()`.
2. Added a new API called `CyU3PUsbResetEndpointMemories()` to reset the endpoint memory block on the FX3 as a whole. This API should only be used to recover from error conditions that cause data transfers through FX3 to freeze.

Tool information

3. Fixed race conditions in the drivers that could cause EP0 transfers (`CyU3PUsbSendEP0Data` or `CyU3PUsbGetEP0Data`) to be failed with the CY_U3P_ERROR_ABORTED code, even if the actual transfer on the bus has passed.
4. Fixed a driver defect that prevented the generation of the CY_U3P_USB_EVENT_CONNECT event when the firmware is configured to enable USB 3.0 connections, and the device is connected to a USB 2.0 host.
5. Updated the USB driver to improve CPU responsiveness when there is a high frequency of U1 power mode entry requests from the host. The earlier implementation was leading to some failures when testing against the host drivers on Windows 8.
6. Added a new USB event (CY_U3P_USB_EVENT_LNK_RECOVERY) for notification of cases where the FX3 device is entering USB 3.0 link recovery.
7. Added new APIs called `CyU3PUsbSetEpPktMode()`, `CyU3PUsbGetBooterVersion()`, and `CyU3PUsbGetErrorCounts()`. Please see the API guide for details on these functions.

3.10.3 DMA driver and APIs

1. Added a new API called to `CyU3PDmaEnableMulticast()`. It is required to call this API before creating any multicast DMA channels.

3.10.4 GPIO driver and APIs

1. Fixed a GPIO driver defect that caused the ARM CPU to get stuck when a GPIO interrupt is raised.
2. Modified the GPIO APIs to allow the state of FX3 GPIOs to be retained when switching from the boot firmware to the full firmware and back.
3. Updated the driver code to keep all FX3 GPIOs unaffected when the device is placed in suspend mode. Earlier, all GPIOs would have been tri-stated when the device was placed in suspend mode.

3.10.5 Two-Stage boot firmware library

1. Added a new API (`CyFx3BootRetainGpioState`) to allow the state of FX3 GPIOs to be retained when switching control from the boot firmware to the full firmware.
2. Added new APIs (`CyFx3BootUsbLPMDisable` and `CyFx3BootUsbLPMEnable`) to dynamically enable/disable U1/U2 power mode acceptance by the boot firmware.
3. Updated the USB driver to handle U1/U2 mode transitions in the middle of control endpoint transfers more efficiently.
4. Updated the build settings for the boot application example to reduce memory footprint of the binary.

3.10.6 Firmware examples

1. Updated the FX3 Boot App example and the USBBulkSourceSink examples to demonstrate switching between the boot firmware and the full firmware.
2. Added a new example called USBIsoSource that demonstrates a high performance ISO IN pipe in parallel with USB control transfers.
3. Enabled the data cache in all firmware examples.

3.10.7 Firmware binary converter

1. Updated the elf2img converter to support output images with section sizes broken down to 2KB and 1KB. This can be used as a work-around for firmware download failures on the Etron USB 3.0 host controller. The Etron host controller (driver) has a known error in the handling of Control OUT transfers with data length of 4 KB, and this leads to firmware download errors. These errors can be worked around by using the elf2img converter with the argument “-i2cconf 0x02” or “-i2cconf 0x00”. This change forces the Control Center to use smaller bursts (2KB or 1KB) during the firmware download process.

3.10.8 SDK documentation

1. Added a new section on FX3 Programming Guidelines to the “Getting Started with FX3 SDK” document.

3.11 Changes from SDK 1.1.1 to SDK 1.2

The FX3 SDK 1.2 release has the following additional changes on top of the 1.1.1 patch version.

3.11.1 Serial peripheral drivers

1. The serial peripheral drivers have been made open source in response to several requests to allow customization of these drivers. The output from these drivers will be part of a different library, which will be linked into the firmware application. Full documentation for all serial peripheral interface registers is also provided as part of the programmer’s manual.

   Please see the **<INSTALL_ROOT>/firmware/lpp_source** folder for the source code for these drivers and APIs. An eclipse project that builds these files into a library (cyu3lpp.a) is also provided in the package.

3.11.2 Device APIs

1. Added support in the firmware to jump back to the boot firmware image, without disconnecting the USB connection. This feature allows users to modify the application code without breaking the USB connection.
2. Added support for all devices (CYUSB3011, CYUSB3012, CYUSB3013 and CYUSB3014) in the FX3 device family. The firmware drivers detect the device part number and return appropriate error codes when unsupported functionality is exercised.

3.11.3 GPIF driver and APIs

1. Added separate types for P-port interface errors and GPIF specific errors reported through the CYU3P_PIB_INTR_ERROR callback. Macros to retrieve the relevant error information from the callback argument have been added.

3.11.4 USB driver and APIs

1. Reorganized the USB driver code so that only applications that make use of the USB host and/or OTG features will include the corresponding driver functions. This change reduces the memory footprint of applications that do not use USB host or OTG features.
2. Updated the USB 3.0 driver to configure the USB block to make endpoint under-run errors on the IN path unlikely. Also, added a new USB event type (CY_U3P_USB_EVENT_EP_UNDERRUN) which is sent to the application if an under-run condition is detected.
3. Added the capability to log USB related events into a user provided buffer. Please see the `CyU3PUsbInitEventLog()` and `CyU3PUsbGetEventLogIndex()` APIs for details.
4. Fixed a FX3 device issue, which caused the device to re-enumerate when doing high throughput data transfers with some USB 3.0 hosts. This error was seen most often when testing with the Intel XHCI host controllers.

3.11.5 Build settings

1. Updated the FX3 example makefiles to function on non-Windows platforms like Linux and Mac. This change allows the examples to be built in batch mode without invoking the Eclipse IDE.
2. Modified the cyfxtx.c file in the SDK to reserve the last 32 KB of RAM for the boot firmware application. This change reduces the amount of memory available for DMA buffers. If the application does not use the 2-stage boot feature, the upper limit of the buffer heap can be restored to the address 0x80000.

3.11.6 Two-Stage boot firmware library

1. Added code to support switch back to the boot application from the main firmware application.
2. Added watchdog timer support. Please see the `CyFx3BootWatchdogConfigure()` API for details.
3. Added a parameter to the `CyFx3BootDeviceInit` API to select between a system clock setting of 384 MHz and 403.2 MHz.
4. Updated the USB boot application code to handle requests for non-existent descriptors cleanly.

3.11.7 Firmware examples

1. Added new Slave FIFO examples in both synchronous and asynchronous modes, that makes use of all the USB endpoints supported by the FX3 device. The 5-bit addressed mode of the Slave FIFO protocol is used so that a separate pipe and DMA channel on the GPIF II side can be associated with each USB endpoint.
2. Added a new example that implements a mass storage device using a portion of the FX3 device RAM.
3. Added a new example that implements a USB Audio Class (UAC) microphone device and streams audio data that it reads from a SPI flash device connected to FX3. This implementation only works at USB 2.0 speeds, as the UAC class does not require the bandwidth provided by USB 3.0.
4. Updated the UsbBulkSourceSink example to make use of the USB event log support.

3.12 Changes from SDK 1.1 to SDK 1.1.1

In addition to the above changes, the SDK 1.1.1 patch version had the following changes on top of the SDK 1.1 release.

3.12.1 General changes

1. A new member called setSysClk400 has been added to the CyU3PSysClockConfig_t structure passed to the `CyU3PDeviceInit()` function. This parameter is used to select whether the FX3 system clock should be running at 384 MHz or 403.2 MHz. The system clock frequency was compulsorily set to 403.2 MHz in previous SDK releases.

3.12.2 USB driver and API changes

1. Added a new USB event called CY_U3P_USB_EVENT_EP0_STAT_CPLT to provide notification of the completion of the status handshake for USB control requests that are handled by the user's callback function.
2. Added a new USB API called `CyU3PUsbDoRemoteWakeup()` to cause FX3 to perform USB 2.0 remote wakeup signaling.
3. Added a new USB API called `CyU3PUsbForceFullSpeed()` to force the FX3 device to function at USB full speed instead of high speed.
4. Added a new USB API called `CyU3PUsbSendDevNotification()` to allow sending of Device Notification Transaction Packets (TP) such as Function Wake and Latency Tolerance Message.
5. Added new USB APIs called `CyU3PUsbLPMDisable()` and `CyU3PUsbLPMEnable()` to disable/enable U1/U2 power mode entry at runtime. Please refer to the API description in the API guide for usage restrictions.
6. Fixed a defect in the USB driver that could cause control transfers without a data phase to fail if the FX3 device was placed in U1/U2 before the status handshake is completed.
7. Made USB driver changes to fix USB 3.0 hotplug errors and TD 9.23 (Reset Device Test) failures in the USBCV suite. These errors had been seen only on some FX3 boards, and were associated with the device failing to go through USB 3.0 link training.
8. Made USB driver changes to improve firmware responsiveness while working under aggressive USB link power management.

9. Updated the USB driver to ensure that the sequence numbers (or data toggles) on all endpoints are cleared when handling a SET_CONFIGURATION request.

3.12.3 I2C driver changes

1. Fixed a defect in the I2C driver, which could cause the `CyU3PI2cTransmitBytes()` API to incorrectly return ***CY_U3P_ERROR_LOST_ARBITRATION*** even in some success cases.

3.12.4 GPIF driver and API changes

1. Fixed a defect in the `CyU3PGpifSMSwitch()` API, which would cause state machine switches to fail if the GPIF designs use mirror states.

3.12.5 Debug API changes

1. Fixed a defect in the `CyU3PDebugPrint()` API when trying to print messages whose total length exceeds 128 characters. The new implementation is limited to a total length of 256 characters, and will return an error if longer messages are passed in as parameters.

3.12.6 Example application changes

1. Updated all example applications to use 32-byte aligned buffers for descriptors and other data that is transferred via DMA to the USB host. This is required for the applications to work when the data cache is enabled.
2. Updated the descriptors and callback function implementations in all USB device mode examples to pass USB chapter 9 compliance tests.
3. Updated the cyfxbulksrcsink example application with additional code demonstrating new features such as link power management control.
4. Updated the GPIF descriptors used in the Slave FIFO examples to the latest version generated by the GPIF II Designer tool.

3.12.7 Boot firmware changes

1. Updated boot firmware library and USB boot example to pass USB Chapter 9 compliance tests and Physical/Link Layer tests.

3.13 Changes from SDK 1.0.1 to SDK 1.1

The SDK 1.1 release of the FX3 SDK has the following changes from the 1.0.1 version.

3.13.1 General changes

1. Macro definition for version information has also been added.
2. FX3 release configuration has reduced error checks for better performance.
3. CyU3PSysEnterSuspendMode API signature has been changed to accommodate the polarity of the wakeup source.
4. CyU3PDebugSysMemInit and CyU3PDebugSysMemDeInit APIs have been added to do debug logging into system memory buffer.
5. CyU3PDebugEnable and CyU3PDebugDisable API signature has been modified to take the thread/module information as a bit mask. CyU3PDebugIsEnabled API has been removed as this is redundant.

3.13.2 USB driver and APIs

1. USB host (cyu3usbhost.h) and OTG (cyu3usbotg.h) APIs have been added.
2. CY_U3P_USB_EVENT_SOF_ITP event has been added to track SOF/ITP USB events. CyU3PUsbEnableITPEvent API has been added to control this.
3. OTG descriptor, CY_U3P_USB_SET_OTG_DESCR, has been added to the list of supported standard USB descriptors.

3.13.3 I2C driver and APIs

1. Error return codes for I2C APIs have been modified to return detailed error information.
2. CyU3PI2cGetErrorCode API has been added to retrieve the error information when an I2C API returns CY_U3P_ERROR_FAILURE.

3.13.4 GPIO

1. GPIF CTL4 is now made available as GPIO and need not be overridden.
2. GPIO register definition (gpio_regs.h) has been included with the release.

3.13.5 Two-Stage boot firmware library

1. The timeout value for I2C and SPI DMA transfer functions have been reduced to multiples of 10 µs (from 100 µs).

3.13.6 Firmware examples

1. cyfxbulklpautoenum and cyfxbulksrcsink examples have been modified to pass USB compliance. Other examples do not have these changes. The changes required are:
   a. USB version field in CyFxUSB20DeviceDscr should be 2.10.
   b. LPM support needs to be enabled in supported device-level features field in CyFxUSBBOSDscr.
   c. Handle SET_FEATURE(FUNCTION_SUSPEND) and CLEAR_FEATURE(FUNCTION_SUSPEND) USB setup requests. These should be allowed to pass if the device is in configured state and failed otherwise.
   d. Register a callback to handle LPM requests and return CyTrue for all requests.
2. Performance optimization changes for cyfxbulksrcsink and cyfxisosrcsink examples have been limited to few macro changes.
3. All ISO examples have been modified to use EP3 IN and EP3 OUT so that mult field can be used.
4. cyfxbulklpmanual example has been modified to invert all bits on the received data.
5. cyfxbulklpauto_cpp, Cyfxbulklplowlevel, cyfxbulklpman_addition, cyfxbulklpman_removal, cyfxbulklpotg, cyfxusbdebug, cyfxusbhost, cyfxusbotg, cyfxusbspigpiomode, cyfxusbi2sdmamode and cyfxuvcinmem_bulk examples have been added.

3.14 Changes from SDK 1.0 to SDK 1.0.1

The SDK 1.0.1 patch release has the following changes from the 1.0 version.

3.14.1 Linux support

1. Linux is now supported as a development and debug platform for FX3 firmware. Please refer to the *FX3_SDK_Linux_Support.pdf* document in the doc folder for installation and usage instructions on a Linux platform.

3.14.2 USB driver

1. Included USB driver changes to fix rare USB 3.0 failures when operating behind a Via SuperSpeed hub. No API interface changes are involved in this.
2. Fixed a defect in the `CyU3PUsbStart()` API to reset the signature and length field used for the USB No Re-enumeration feature of the 2-Stage boot firmware

3.14.3 GPIF driver and APIs

1. Fixed a defect in the `CyU3PGpifDisable()` API when the forceReload parameter is set to True.
2. All optional PIB interrupts are kept disabled by default and are only turned on when the `CyU3PPibRegisterCallback()` function is used to enable callbacks for these events.
3. Added the `CyU3PPibSelectIntSources()` API to select the sources that can drive the output INT pin from the FX3 device to the external processor/device.

3.14.4 I2C driver and APIs

1. Fixed a problem in the `CyU3PI2cTransmitBytes()` API where no error is reported for a single byte transfer even if the target device is missing.

3.14.5 UART driver and APIs

1. Fixed a defect in the UART driver that led to UART callbacks never being invoked.

3.14.6 ELF to boot image converter

1. Fixed a defect that caused boot failure when the boot image was stored across multiple I2C EEPROM devices.

3.14.7 Two-Stage boot firmware library

1. Fixed a defect in the boot support firmware library which caused the no re-enumeration after firmware load feature to intermittently fail.
2. Reduced internal processing delays in the SPI driver to get better SPI transfer performance.
3. Added Full Speed support in the usb_boot.c
4. Fixed a defect in usb_boot.c, which caused the descriptor length to be calculated incorrectly.

3.14.8 Firmware examples

1. Fixed un-initialized variable errors in the cyfxflashprog, cyfxusbi2cdmamode, and cyfxusbspidmamode examples that would cause write transfers to fail on some builds.
2. Updated the cyfxusbi2cregmode example to enable reading/writing arbitrary counts of data.
3. Fixed a condition check in the cyfxbulklpautoenum example that caused errors in retrieving string descriptors.
4. Fixed errors in the build settings for the cyfxbulklpmanual_rvds example, and added a functional release configuration.

4 Known problems and limitations of FX3 SDK 1.3.5

This FX3S SDK 1.3.5 release has the following known issues and limitations:

1. The settings for DMA channels associated with USB sockets are likely to be invalidated as part of the USB device mode connection startup. Therefore, it is required that all USB related DMA channels be reset (CyU3PDmaChannelReset) and the re-enabled after each enumeration cycle. This can be done on a USB reset event, or on a Set Configuration event; as none of the USB DMA channels are expected to be used before SET_CONFIGURATION is completed.
2. When there is a failure in an I2C transaction (a DMA failure or an API return of CY_U3P_ERROR_BLOCK_FAILURE), the I2C block needs to be reset. This can be done by invoking `CyU3PI2cDeInit()`, followed by a `CyU3PI2cInit()` and a `CyU3PI2cSetConfig()`. In multi-master use cases, the `CyU3PI2CSendStopCondition()` API can be used to ensure that FX3 has driven out a stop condition at the end of a failed transfer.
3. After a `CyU3PI2cReceiveBytes()/CyU3PI2cTransmitBytes()` API call, sufficient time must be given for the slave to complete the operation before the next transaction (read/write) is issued. This can be achieved by using the `CyU3PI2cWaitForAck()` or by providing a sufficient delay.
4. When the warm reset functionality of the `CyU3PDeviceReset()` API is used, any global variables used by the application will not be properly re-initialized. This is because the startup code that initializes these variables would have been lost and cannot be executed again without loading the application again. If warm reset is needed, the application code must ensure that all necessary data is saved and restored or re-initialized as required.
5. The `CyU3PSpiWaitForBlockXfer()` function waits forever if there are no pending transfers at the time of calling this API.
6. Any UART/I2C/SPI read transfers in DMA mode that do not fill up the entire DMA buffer will not trigger a DMA callback or a transfer complete event. The application needs to check for transfer completion based on the UART/I2C/SPI events and then invoke the `CyU3PDmaChannelSetWrapUp()` API on the DMA channel.
7. ARM GCC tool chain, if any, installed on the host system needs to be completely removed before installing the ARM GCC toolchain supplied with this package. If the older installation is not removed, it could lead to firmware build issues.
8. Some data transfer errors may be seen if a Zero Length Packet is followed very quickly (within one microframe or 125 us) by another data packet on a burst enabled USB IN endpoint operating at super speed. The solution is to ensure that some time is allowed to elapse between a ZLP and the next data packet on burst enabled USB IN endpoints. If this cannot be ensured at the data source, the `CyU3PDmaChannelSetSuspend()` API can be used to suspend the corresponding USB DMA socket on seeing the EOP condition. The channel operation can then be resumed as soon as the suspend callback is received.
9. The RTOS porting layer code that is part of the cyfxtx.c file in the firmware examples has a tight correlation with the memory map information specified in the linker script file (fx3.ld). When moving to a newer version of the FX3 SDK using an example that was previously created, it is required to copy the new version of the cyfxtx.c file (from the **<FX3_INSTALL_ROOT>/firmware/common** folder) into the project.
10. The CyU3PGpifSMSwitch API may not be able to trigger a desired state machine switch if the state machine in use makes use of mirror states. This is because the state machine may actually be stuck in a mirror of the user specified start state instead of in the start state itself. In this release, the API has been updated to trigger an immediate switch if the state machine is in the specified start state or one of its mirrors. This will not be sufficient if the state machine reaches a mirror of the start state only after the Switch API has been called.

    If the `CyU3PGpifSMSwitch` API needs to be used in systems which make use of state machines with mirror states, it is recommended that the application specify a timeout value in the `CyU3PGpifSMSwitch()` API call and retry the switch operation periodically until it succeeds.

Known problems and limitations of FX3 SDK 1.3.5

11. The gcc linker script provided with the FX3 SDK (**firmware/common/fx3.ld**) does not initialize a runtime heap that is required for standard C library functions such as sprintf. If you want to use these functions,you mustovide an implementation for standard system calls and update the fx3.ld file to create a heap. Please refer to the cyfxbulklpauto_cpp example for a sample implementation of the required system calls, and the fx3cpp.ld file for a script sample with heap initialization.
12. When the FX3 device is functioning as a high-speed USB device with high bandwidth isochronous endpoints, the PID sequence of the ISO data packets is governed solely by the isomult setting. The length of the data packet is not considered while generating the PID sequence during each microframe. For example, even if a short packet is being sent on an endpoint with MULT set to 2, the PID used will be DATA2.

    The workaround for this problem is to reconfigure the endpoint with a lower isomult setting prior to sending short packets, and then switching back to the original value. Please refer to the USBVideoClass firmware example for an implementation of this workaround.
13. The 5-bit Slave FIFO descriptors provided in the FX3 SDK and the GPIF II Designer cause the FX3 device to flag GPIF errors during data transfers. These errors are false errors and can safely be ignored. The application can avoid registering for the CYU3P_PIB_INTR_ERROR callback if these interrupts and callbacks are found to be slowing down the system.
14. If the clkCfg->setSysClk400 parameter to the `CyU3PDeviceInit` API is set to true, the operating clock frequency of the FX3 device will be changed at runtime. This can cause a JTAG debug session using the JLINK debugger to break. This problem can be prevented by setting up the FX3 device clocks to the desired value during JTAG initialization itself. If the clocks have already been set up to the desired value, the FX3 API does not make any further changes. Please add the following code fragment to the set of Initialize commands used in Eclipse-based debug configuration.

```
# If required, set the FX3 system clock to faster than 400 MHz.
# This is done here to prevent debug session errors due to the
# clock bein g changed at runtime.
# Update with the correct value from list below.
# Clock input is 19.2 MHz: Value = 0x00080015
# Clock input is 38.4 MHz: Value = 0x00080115
monitor memU32 0xE0052000 = 0x00080015
# Add a delay to let the clock stabilize.
monitor sleep 1000
```

15. USB data transfers through the FX3 device may get stuck if the DMA channels corresponding to the endpoints are unexpectedly reset. This condition can be detected by checking for a CY_U3P_USB_EVENT_EP_UNDERRUN event. The under-run error can be avoided by ensuring that data transfer through the endpoint is disabled before the DMA channel is reset. This is done by using the `CyU3PUsbSetEpNak()` to place the endpoint in NAK state for about 100 µs, before resetting the DMA channel.

```
e.g.: CyU3PUsbSetEpNak (CY_FX_EP_CONSUMER, CyTrue);
      CyU3PBusyWait (100);
      CyU3PDmaChannelReset (&glEpSourceChannel);
      CyU3PUsbSetEpNak (CY_FX_EP_CONSUMER, CyFalse);
```

    Resetting an USB endpoint using the `CyU3PUsbResetEp()` API causes the endpoint to lose all state. If the endpoint was in a flow control state (FX3 has sent an NRDY and is expected to send ERDY when data is ready) at the time of doing a reset, the ERDY may not be sent even in the endpoint (DMA channel) becomes ready after the reset. This can be corrected by manually triggering an ERDY TP from FX3 using the

Known problems and limitations of FX3 SDK 1.3.5

`CyU3PUsbSendErdy()` API after the reset is done. The ERDY can be sent immediately regardless of whether the endpoint is in a ready state, and the endpoint state will thereafter be tracked correctly by the device.

5 Technical support

For further assistance, go to **Infineon Community Support**.

6 Additional information

For more information about the FX3 product family, visit **FX3 Product page**.

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.

Edition 2022-08-18

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2022 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Go to www.infineon.com/support

Document reference

002-32817 Rev. *A

IMPORTANT NOTICE

The information given in this document shall in no event be regarded as a guarantee of conditions or characteristics (“Beschaffenheitsgarantie”) .

With respect to any examples, hints or any typical values stated herein and/or any information regarding the application of the product, Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind, including without limitation warranties of non-infringement of intellectual property rights of any third party.

In addition, any information given in this document is subject to customer’s compliance with its obligations stated in this document and any applicable legal requirements, norms and standards concerning customer’s products and any use of the product of Infineon Technologies in customer’s applications.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer’s technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology, delivery terms and conditions and prices please contact your nearest Infineon Technologies office (**www.infineon.com**).

WARNINGS

Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies’ products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.